Module 4

Module 4 (Prediction and Control)

Dynamic Programming - Policy Evaluation (Prediction), Policy Improvement, Policy Iteration, Value Iteration.

Monte Carlo (MC) for model free prediction and control - Monte Carlo Prediction, Monte Carlo Estimation of Action Values, Monte Carlo Control, Monte Carlo Control without Exploring Starts, Off-policy Prediction via Importance Sampling, Incremental Implementation, Off-policy Monte Carlo Control.

1. What is Dynamic Programming (DP)?

- **Dynamic Programming (DP)** refers to a **class of algorithms** used to compute **optimal policies and value functions** when:

  - The environment is **fully known**
  - The problem can be modeled as a **Markov Decision Process (MDP)**
  - DP solves complex decision problems by **breaking them into smaller subproblems** and solving them **recursively** using **Bellman equations**.

- Dynamic means The problem changes step by step.
- Decisions affect future outcomes.
- In RL:
  - You take an action.
  - You move to a new state.
  - That affects future rewards.
  - So the problem is dynamic (changing over time).

- In RL, Dynamic Programming means:
- Using the Bellman equation to compute the best value function and policy,assuming we know the full model of the environment.
- That means we know:
  - All states
  - All actions
  - All rewards
  - Transition probabilities $p(s', r' \mid sa)$
- Without knowing this model , we cannot use classical DP.

Value now

= Reward now + Discounted value later

This recursive idea is the heart of DP.

Key Idea of DP

- The central idea is:
- Use **value functions** to search for good policies.
- Once we compute:

  $v^*(s)$ optimal state value function
  OR
  q^*(s,a) optimal action-value function)

- We can directly get the optimal policy.

$$
\begin{aligned}
v_*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \max_a \sum_{s',r} p(s', r|s, a)\Big[r + \gamma v_*(s')\Big] \qquad (4.1)
\end{aligned}
$$

or

$$
\begin{aligned}
q_*(s, a) &= \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a\Big] \\
&= \sum_{s',r} p(s', r|s, a)\Big[r + \gamma \max_{a'} q_*(s', a')\Big], \qquad (4.2)
\end{aligned}
$$

for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, and $s' \in \mathcal{S}^+$. As we shall see, DP algorithms are obtained by turning Bellman equations such as these into assignments, that is, into update rules for improving approximations of the desired value functions.

Role of DP in RL

- Dynamic Programming provides the theoretical foundation of reinforcement learning by defining Bellman equations and offering exact methods (value iteration and policy iteration) to compute optimal value functions and policies in known MDPs.
- In RL, DP:

✓ Defines Bellman equations
✓ Computes exact optimal solutions (when model known)
✓ Introduces policy evaluation
✓ Introduces policy improvement
✓ Forms the base of value iteration & policy iteration
✓ Provides theoretical guarantees

Note:

- DP assumes full knowledge of model
  ✓ Works on finite MDPs
  ✓ Uses value functions
  ✓ Based on Bellman optimality equations
  ✓ Recursive structure
  ✓ Used to compute optimal policy
  ✓ Theoretical foundation of RL

Main Dynamic Programming Algorithms

1. Policy Evaluation

- Computes $v_\pi(s)$for a **fixed policy**
- Uses Bellman **expectation** equation
- Iterative process

2.Policy Improvement

- Improves a policy using the computed value function
- Chooses greedy actions

3. Policy Iteration

Repeats:

- Policy Evaluation
- Policy Improvement
- Until the policy stops changing.

4. Value Iteration

- Combines evaluation + improvement into **one step**
- Faster than policy iteration
- Uses Bellman **optimality** equation directly

Method	Theory Role
Policy Evaluation	Solves Bellman expectation equation
Policy Improvement	Solves greedy maximization
Policy Iteration	Alternates between both
Value Iteration	Combines both into one operator

******

We want the agent to **learn the best policy**.

Policy means:

👉 **Which action to take in each state**

Example:

State = **S**

Actions:

```
Left
Right
Up
Down
```

The agent must learn **which action gives the highest reward**.

Policy Evaluation

1. Policy Evaluation

First we consider how to compute the state-value function v_π for an arbitrary policy π. This is called *policy evaluation* in the DP literature. We also refer to it as the *prediction problem*. Recall from Chapter 3 that, for all $s \in \mathcal{S}$,

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s\big] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \qquad (4.3) \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\Big[r + \gamma v_\pi(s')\Big], \qquad (4.4) \end{aligned}$$

where $\pi(a|s)$ is the probability of taking action a in state s under policy π, and the expectations are subscripted by π to indicate that they are conditional on π being followed. The existence and uniqueness of v_π are guaranteed as long as either $\gamma < 1$ or eventual termination is guaranteed from all states under the policy π.

Iterative Policy Evaluation process*

- Start with any guess for state values.
- Use Bellman equation to improve the guess.
- Repeat many times.
- The values will eventually become correct.
- When they stop changing → we found v_π.

We repeatedly apply:

- Value = Reward + Discounted Future Value

until the numbers stop changing.

- **The Bellman Update Rule**

$$v_{k+1}(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s \right]$$

i.e to compute the new value of state s:Use the **old values** v_k and apply the Bellman expectation equation.

Expanded Version

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_k(s') \right]$$

Use v_k Instead of v_{k+1}?
Because:
We are improving the value step by step.
We use old estimates to compute new ones.
That's why it is called **Iterative Policy Evaluation**

- Bellman equation says:If you already know the correct value function, it must satisfy this formula.
- Bellman update rule says:'Use this formula repeatedly to find the correct value.'
- In practice, we don't know v_π.

So we approximate it.

- We use:

$$v_{k+1}(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s]$$

'

- Left side → new value
- Right side → old value

This is an **algorithmic update step**.

- We use it repeatedly until values stop changing.

The Bellman equation defines the exact recursive relationship satisfied by the true value function, whereas the Bellman update rule is the iterative computational procedure used to approximate that value function.

Input π, the policy to be evaluated
Initialize an array $V(s) = 0$, for all $s \in \mathcal{S}^+$
Repeat
    $\Delta \leftarrow 0$
    For each $s \in \mathcal{S}$:
        $v \leftarrow V(s)$
        $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma V(s')\right]$
        $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)
Output $V \approx v_\pi$

Figure 4.1: Iterative policy evaluation.

Problem:

Consider a **2×2 Gridworld** with 4 states:

$$\begin{matrix} S_1 & S_2 \\ S_3 & S_4 \end{matrix}$$

- S_4 is a **terminal state**.
- From all non-terminal states, actions = {Up, Down, Left, Right}.
- If an action tries to move outside the grid, the agent remains in the same state.
- Reward = **–1 for every move**
- Discount factor $\gamma = 1$
- Policy = **Equiprobable random policy** (each action probability = 1/4)

Use **Iterative Policy Evaluation** to compute the value function $v_\pi(s)$ for all states.

Solution:

- Bellman Update Rule

Expanded Version

$$v_{k+1}(s) = \sum_{a} \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_k(s')\right]$$

Since $\frac{1}{4}$ is constant (same for all actions), we can factor it out.

Expanded Version

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\,[r + \gamma v_k(s')]$$

In this problem:

There are **4 actions**.

Equiprobable random policy means:

$$\pi(a_1|s) = \frac{1}{4}$$

$$\pi(a_2|s) = \frac{1}{4}$$

$$\pi(a_3|s) = \frac{1}{4}$$

$$\pi(a_4|s) = \frac{1}{4}$$

$$\sum_a \pi(a|s) = \frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}$$

$$= \frac{4}{4}$$

$$= 1$$

The Bellman update is:

$$\sum_a \pi(a|s) X(a)$$

$$X(a) = \sum_{s',r} P(s',r|s,a)\,[r + \gamma v_k(s')]$$

Since each π(a|s) = 1/4:

$$= \frac{1}{4}X(a_1) + \frac{1}{4}X(a_2) + \frac{1}{4}X(a_3) + \frac{1}{4}X(a_4)$$

Factor out 1/4:

$$= \frac{1}{4}\left[X(a_1) + X(a_2) + X(a_3) + X(a_4)\right]$$

Discount factor $\gamma = 1$

$$v_{k+1}(s) = \frac{1}{4}\sum_{a}\left[-1 + v_k(s')\right]$$

In our 2×2 example:

- If you take action Right from S_1, you go to S_2 with certainty.
- If you hit a wall, you stay in the same state with certainty.
- Reward is always −1.

So transitions are **deterministic**.

That means:

$$P(s', r|s, a) = \begin{cases} 1 & \text{for the actual next state} \\ 0 & \text{for all others} \end{cases}$$

Bellman Update Rule

$$v_{k+1}(s) = \frac{1}{4}\sum_{a}[-1 + v_k(s')]$$

Iteration 0 (Initialization)

$$v_0(S_1) = 0, \quad v_0(S_2) = 0, \quad v_0(S_3) = 0, \quad v_0(S_4) = 0$$

Compute Iteration 1

We compute $v_1(s)$ using v_0.

Since all $v_0 = 0$, things will be simple.

1. **<u>State S_1</u>**

   Think carefully: from S_1

   - Up → stays in S_1
   - Left → stays in S_1
   - Right → goes to S_2
   - Down → goes to S_3

   So there are 4 actions total.

   Now apply formula:

$$v_1(S_1) = \frac{1}{4}\Big[(-1 + v_0(S_1)) + (-1 + v_0(S_1)) + (-1 + v_0(S_2)) + (-1 + v_0(S_3))\Big]$$

Now substitute values:

All v_0 are 0.

So:

$$= \frac{1}{4}\Big[(-1+0)+(-1+0)+(-1+0)+(-1+0)\Big]$$

Each term equals −1.

So:

$$= \frac{1}{4}[-1-1-1-1]$$

$$= \frac{-4}{4}$$

$$= -1$$

So:

$$v_1(S_1) = -1$$

State s2

From S_2:

- Up → stays in S_2
- Right → stays in S_2
- Left → goes to S_1
- Down → goes to S_4 (terminal)

Now apply formula:

$$v_1(S_2) = \frac{1}{4}\Big[(-1 + v_0(S_2)) + (-1 + v_0(S_2)) + (-1 + v_0(S_1)) + (-1 + v_0(S_4))\Big]$$

Substitute zeros:

$$= \frac{1}{4}[-1 - 1 - 1 - 1]$$

$$= \frac{-4}{4}$$

$$= -1$$

So:

$$v_1(S_2) = -1$$

State s3

From S_3:

- Left → stays
- Down → stays
- Up → to S_1
- Right → to terminal S_4

Same structure as S_2.

So:

$$v_1(S_3) = -1$$

State s4

Terminal value never changes:

$$v_1(S_4) = 0$$

Final Result After Iteration 1

$$\begin{matrix} -1 & -1 \\ -1 & 0 \end{matrix}$$

After **Iteration 1**, we computed:

$$v_1 = \begin{matrix} -1 & -1 \\ -1 & 0 \end{matrix}$$

So:

$$v_1(S_1) = -1$$

$$v_1(S_2) = -1$$

$$v_1(S_3) = -1$$

$$v_1(S_4) = 0$$

Iteration 2

State S_1

First recall transitions from S_1:

- Up → stays in S_1
- Left → stays in S_1
- Right → goes to S_2
- Down → goes to S_3

Now apply formula:

$$v_2(S_1) = \frac{1}{4}\Big[(-1 + v_1(S_1)) + (-1 + v_1(S_1)) + (-1 + v_1(S_2)) + (-1 + v_1(S_3))\Big]$$

Now substitute values from Iteration 1:

$$v_1(S_1) = -1$$

$$v_1(S_2) = -1$$

$$v_1(S_3) = -1$$

$$v_2(S_1) = -2$$

State S2

Recall transitions from S_2:

- Up → stays in S_2
- Right → stays in S_2
- Left → goes to S_1
- Down → goes to terminal S_4

Now write formula:

$$v_2(S_2) = \frac{1}{4}\Big[(-1 + v_1(S_2)) + (-1 + v_1(S_2)) + (-1 + v_1(S_1)) + (-1 + v_1(S_4))\Big]$$

Now substitute values:

$$v_1(S_2) = -1$$

$$v_1(S_1) = -1$$

$$v_1(S_4) = 0$$

$$v_2(S_2) = \frac{-7}{4}$$

$$= -1.75$$

State S3

Transitions from S_3:

- Left → stays
- Down → stays
- Up → to S_1
- Right → to terminal S_4

This is symmetric to S_2.

So:

$$v_2(S_3) = -1.75$$

State s4

Terminal stays:

$$v_2(S_4) = 0$$

In policy evaluation, after each iteration we compute:

$$\Delta = \max_{s} \left| v_{k+1}(s) - v_k(s) \right|$$

For every state

Take the absolute difference between new value and old value

Δ is the **largest** of those differences

Because we stop when:

$$\Delta < \theta$$

where θ (theta) is a very small number like:

$$\theta = 0.01 \quad \text{or} \quad 0.001$$

That means values are not changing much anymore → convergence.

Compute Δ for Iteration 2

We already have:

Iteration 1:

$$v_1 = \begin{matrix} -1 & -1 \\ -1 & 0 \end{matrix}$$

Iteration 2:

$$v_2 = \begin{matrix} -2 & -1.75 \\ -1.75 & 0 \end{matrix}$$

$$\{1,\ 0.75,\ 0.75,\ 0\}$$

Largest value =

$$\Delta = 1$$

Should we stop?

Suppose:

$$\theta = 0.01$$

Check:

$$\Delta = 1$$

Is:

$$1 < 0.01?$$

No ❌

So we **continue**.

Policy Improvement

Policy Improvement

Suppose:

At state s:

- Following policy gives value:

$$v_\pi(s) = 10$$

Now we check another action a:

- If we take a once and then follow π:

$$q_\pi(s, a) = 14$$

What does that mean?

- The policy is making a mistake at state s.

- There exists a better action.

So we improve the policy.

The current policy's action is already included inside $v_\pi(s)$.

Important idea

In fact:

$$v_\pi(s) = q_\pi(s, \pi(s))$$

π(s) -The action chosen by policy π in state s.

This is VERY important.

So we are really comparing:

$$q_\pi(s, a) \quad \text{vs} \quad q_\pi(s, \pi(s))$$

$q_\pi(s, a)$ - Take action a in state s and Then follow policy π forever

- We are checking:
- Is there any action better than the one chosen by policy?
- If yes → improve policy.

Policy Improvement Theorem

Let π and π' be two deterministic policies such that:

Equation (4.7)

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \quad \text{for all } s \in S$$

This means:

The action chosen by π' is at least as good as following π, for every state.

Then:

Equation (4.8)

$$v_{\pi'}(s) \geq v_\pi(s) \quad \text{for all } s \in S$$

So the new policy π' is **better or equal** to π.

If strict inequality holds in (4.7) for at least one state,

then strict improvement occurs in (4.8) for at least one state.

We are only testing:

If we change the first action only, is it better?
That's why subscript remains π.

Proof

We start from (4.7):

$$v_\pi(s) \leq q_\pi(s, \pi'(s))$$

Now expand step by step:

$$v_\pi(s) \leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right]$$

The policy improvement assumption says:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \quad \text{for all states } s$$

IMPORTANT:

It holds for **every state**, not just the starting state.

So it must also hold at state S_{t+1}.

So we can write:

$$q_\pi(S_{t+1}, \pi'(S_{t+1})) \geq v_\pi(S_{t+1})$$

Rearranged:

$$v_\pi(S_{t+1}) \leq q_\pi(S_{t+1}, \pi'(S_{t+1}))$$

That is the only thing we are using.

$$v_\pi(s) \leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right]$$

Replace $v_\pi(S_{t+1})$ again using (4.7):

$$\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s\right]$$

Because:

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1})]$$

Expand again:

$$= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s\right]$$

Continue expanding repeatedly:

$$\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s\right]$$

Final result:

$$= v_{\pi'}(s)$$

Thus:

$$v_{\pi'}(s) \geq v_\pi(s)$$

This proves the theorem.

If

$$q_\pi(s, \pi'(s)) \geq v_\pi(s)$$

for all states,

then

$$v_{\pi'}(s) \geq v_\pi(s)$$

Proof (all equation in one place)

- Left side: follow old policy π from start.
- Right side:
  - First take action chosen by new policy π'
  - Then follow old policy π

We start from (4.7):

$$v_\pi(s) \leq q_\pi(s, \pi'(s))$$

Now expand step by step:

$$v_\pi(s) \leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right]$$

Replace $v_\pi(S_{t+1})$ again using (4.7):

$$\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s\right]$$

Expand again:

$$= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s\right]$$

Continue expanding repeatedly:

$$\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s\right]$$

Final result:

$$= v_{\pi'}(s)$$

Thus:

$$v_{\pi'}(s) \geq v_\pi(s)$$

This proves the theorem.

Greedy Policy (important)

The greedy policy with respect to v_π is defined as:

$$\boxed{\pi'(s) = \arg\max_a q_\pi(s, a)}$$

This means:

At each state s,

choose the action that gives maximum $q_\pi(s, a)$.

Since

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right]$$

We can write:

$$\pi'(s) = \arg\max_a \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right]$$

Or in full probability form:

$$\pi'(s) = \arg\max_a \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

Why Does Greedy policy Guarantee Improvement?

$$v_\pi(s) = q_\pi(s, \pi(s))$$

Now greedy policy chooses action that maximizes $q_\pi(s, a)$.

So automatically:

$$q_\pi(s, \pi'(s)) \geq q_\pi(s, \pi(s))$$

Therefore:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s)$$

So the condition of policy improvement theorem is satisfied.

Hence:

$$v_{\pi'}(s) \geq v_\pi(s)$$

Greedy is not changing just one action.

It changes the action at **every state** (if needed).

So greedy action selection across all states = new policy.

Policy Iteration

Once a policy, π, has been improved using v_π to yield a better policy, π', we can then compute $v_{\pi'}$ and improve it again to yield an even better π''. We can thus obtain a sequence of monotonically improving policies and value functions:

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

where $\xrightarrow{\text{E}}$ denotes a policy *evaluation* and $\xrightarrow{\text{I}}$ denotes a policy *improvement*. Each policy is guaranteed to be a strict improvement over the previous one (unless it is already optimal). Because a finite MDP has only a finite number of policies, this process must converge to an optimal policy and optimal value function in a finite number of iterations.

This way of finding an optimal policy is called *policy iteration*. A complete algorithm is given in Figure 4.3. Note that each policy evaluation, itself an iterative computation, is started with the value function for the previous policy. This typically results in a great increase in the speed of convergence of policy evaluation (presumably because the value function changes little from one policy to the next).

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Repeat
   $\quad \Delta \leftarrow 0$
   $\quad$ For each $s \in \mathcal{S}$:
   $\quad\quad v \leftarrow V(s)$
   $\quad\quad V(s) \leftarrow \sum_{s',r} p(s', r|s, \pi(s)) \left[r + \gamma V(s')\right]$
   $\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   $\quad a \leftarrow \pi(s)$
   $\quad \pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V(s')\right]$
   $\quad$ If $a \neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return V and π; else go to 2

Figure 4.3: Policy iteration (using iterative policy evaluation) for v_*. This algorithm has a subtle bug, in that it may never terminate if the policy continually switches between two or more policies that are equally good. The bug can be fixed by adding additional flags, but it makes the pseudocode so ugly that it is not worth it. :-)

Value Iteration

One drawback to policy iteration is that each of its iterations involves policy evaluation, which may itself be a protracted iterative computation requiring multiple sweeps through the state set. If policy evaluation is done iteratively, then convergence exactly to v_π occurs only in the limit. Must we wait for exact convergence, or can we stop short of that? The example in Figure 4.2 certainly suggests that it may be possible to truncate policy evaluation. In that example, policy evaluation iterations beyond the first three have no effect on the corresponding greedy policy.

In fact, the policy evaluation step of policy iteration can be truncated in several ways without losing the convergence guarantees of policy iteration. One important special case is when policy evaluation is stopped after just one sweep (one backup of each state). This algorithm is called *value iteration*. It can be written as a particularly simple backup operation that combines the

- Policy Evaluation
  - Calculate the value of each state for the **current policy**.
  - This usually requires **many iterations** until values stabilize.
  - Iteration updates might look like:V1(s) → V2(s) → V3(s) → V4(s) → … → converged Vπ(s)
  - So we repeat value updates **many times**.
- Policy Improvement
  - After values are computed, we improve the policy:

$$\pi'(s) = argmax_a Q(s,a)$$

  - Then we repeat the process.
- policy iteration

1. Choose a policy
2. Fully analyze it (many iterations)
3. Improve policy

- Value Iteration
- Instead of fully evaluating the policy in policy iteration, we can stop evaluation early. If we stop after just one sweep of all states, the algorithm becomes **Value Iteration**.
- A dynamic programming method where policy evaluation is truncated to a single sweep and values are updated using the Bellman optimality equation to directly compute the optimal value function.(important definition)

- Study the policy iteration and value iteration and their differences (important – see table in next slide)

Difference Between Policy Iteration and Value Iteration

Aspect	Policy Iteration	Value Iteration
Basic Idea	Alternates between **policy evaluation** and **policy improvement**	Directly computes optimal values using the **Bellman optimality equation**
Policy Evaluation	Performed **fully** until the value function converges to v_π	**Truncated** – only **one sweep** of updates is performed
Policy Improvement	Done **after evaluation step**	Done **implicitly during value update** using **max over actions**
Update Equation	Uses **Bellman expectation equation**	Uses **Bellman optimality equation**
Action Selection	Policy determines which action to evaluate	Maximum over all actions is taken
Iteration Structure	1. Evaluate policy repeatedly 2. Improve policy	Single update step combining evaluation and improvement
Convergence Behavior	Fewer iterations but **each iteration expensive**	More iterations but **each iteration simpler**
Final Policy	Policy improves step-by-step	Policy extracted from optimal value function at the end

If we do **only one sweep of evaluation**, the algorithm simplifies into **Value Iteration**.

Because now the update becomes:

$$V(s) = \max_a \sum_{s'} P(s'|s,a)[R + \gamma V(s')]$$

So we directly choose the **best action** while updating the value.

Thus **evaluation and improvement happen together**.

important

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat
$\quad \Delta \leftarrow 0$
$\quad$ For each $s \in \mathcal{S}$:
$\qquad v \leftarrow V(s)$
$\qquad V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$
$\qquad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, π, such that
$\quad \pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$

Figure 4.5: Value iteration.

Introduction to Monte Carlo Methods

Introduction to Monte Carlo Methods

- Learning methods for:
  - Estimating value functions
  - Discovering optimal policies
- Unlike Dynamic Programming (DP):
  - **Do NOT require full knowledge of environment**
  - Learn directly from **experience**

Difference from Dynamic Programming

- Dynamic Programming:
  - Requires full MDP model
  - Computes exact expectations
- Monte Carlo:
  - Requires only sampled experience
  - Uses averages of actual returns
  - No need for full transition probabilities

Monte Carlo Methods

- Model-free learning method
- Learn from complete episodes
- Based on averaging returns
- Works without transition probabilities
- Uses sample experience instead of full model
- Extends DP ideas to real-world learning

Monte Carlo Prediction

- Goal: Learn the **state-value function** $v_\pi(s)$
- Given:
  - A fixed policy π
- Use:
  - Sample episodes generated by following π

This is the **prediction problem** (policy evaluation).

State-value

$$v_\pi(s)$$

Definition:

- Expected return starting from state s
- Return = cumulative discounted future reward

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$$

So:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Core Idea of Monte Carlo Prediction

- Observe returns after visiting state s
- **Average those returns**
- As number of samples increases:

Average → converges to expected value

This is the foundation of Monte Carlo methods.

- We assume:
  - Episodes are generated by following policy π
  - State $s$may appear multiple times in one episode
- Each occurrence of s is called a **Visit to state** s

- An **episode is A complete sequence of interaction from start state to terminal state.**
- It includes:

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$$

- Where:
  - S_0 =starting state
  - A_t =action at time t
  - R_{t+1} =reward received
  - S_T =terminal (final) state
- The episode ends when a **terminal state** is reached.

Two Types of Monte Carlo Prediction

1️⃣ First-Visit Monte Carlo

- Consider only the **first time** state s appears in an episode
- Compute return from that first occurrence
- Average across episodes

$$v_\pi(s) = \text{average of returns after first visits}$$

2️⃣ Every-Visit Monte Carlo

- Consider **every occurrence** of state s in an episode
- Compute return from each occurrence
- Average all returns

$$v_\pi(s) = \text{average of returns after all visits}$$

The First-visit MC method for estimating v.

- Start with guess for values
- Generate episode
- For each state:
  - **Compute return**
  - **Store it**
  - **Update average**
- Repeat
- Estimates improve gradually

Initialize:
  $\pi \leftarrow$ policy to be evaluated
  $V \leftarrow$ an arbitrary state-value function
  $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Repeat forever:
  Generate an episode using π
  For each state s appearing in the episode:
    $G \leftarrow$ return following the first occurrence of s
    Append G to $Returns(s)$
    $V(s) \leftarrow \text{average}(Returns(s))$

Figure 5.1: The first-visit MC method for estimating v_π. Note that we use a capital letter V for the approximate value function because, after initialization, it soon becomes a random variable.

Consider a simple episode under a fixed policy:

States visited in one episode:

$$S_1 \rightarrow S_2 \rightarrow S_1 \rightarrow S_3 \rightarrow Terminal$$

Rewards received:

- From $S_1 \rightarrow S_2$: reward = 2
- From $S_2 \rightarrow S_1$: reward = 3
- From $S_1 \rightarrow S_3$: reward = 4
- From $S_3 \rightarrow Terminal$: reward = 5

Assume:

- Discount factor $\gamma = 1$ (to keep it simple)

Compute Returns

Return G_t = sum of future rewards from that time step.

For the episode:

Time	State	Reward	Return G_t
t=0	S_1	2	2+3+4+5 = 14
t=1	S_2	3	3+4+5 = 12
t=2	S_1	4	4+5 = 9
t=3	S_3	5	5

So:

- First visit of S_1 → return = 14
- Second visit of S_1 → return = 9

First-Visit Monte Carlo: We update a state value **only the first time the state appears in an episode.**

From our episode:

- S_1 → use return **14** (ignore 9)
- S_2 → use 12
- S_3 → use 5

So estimated values after this episode:

$$V(S_1) = 14$$

$$V(S_2) = 12$$

$$V(S_3) = 5$$

If we run more episodes, we take the **average of first-visit returns**.

Every-Visit Monte Carlo: We update using **every time the state appears** in the episode.

Now for S_1:

- First visit → 14
- Second visit → 9

So:

$$V(S_1) = (14 + 9)/2 = 11.5$$

Other states:

$$V(S_2) = 12$$

$$V(S_3) = 5$$

Monte Carlo Estimation of Action Values

In Reinforcement Learning there are two kinds of value functions:

State value

$$v_\pi(s)$$

Expected return when starting from **state s** and following policy π.

Action value

$$q_\pi(s, a)$$

Expected return when starting in **state s**, taking **action a**, then following policy π.

When a **model is available**

A model means we know:

- transition probabilities $P(s'|s,a)$
- reward function

Then **state values are enough**.

Because we can compute:

$$Q(s,a) = R + \gamma V(s')$$

So we can **look one step ahead** and choose the best action.

This is what we did in **Dynamic Programming (DP)**.

When a **model is NOT available**

We **cannot predict the next state**.

So state value alone cannot tell us which action is better.

Example:

State **S**

Possible actions

```
Left
Right
```

If we only know

$$V(S) = 5$$

We still don't know:

```
Is Left better?
Is Right better?
```

Therefore we must estimate

$$q_\pi(s, a)$$

for **each action**.

- Goal of Monte Carlo Control

Main objective:

$$\text{Estimate } q^*(s, a)$$

Why?

Because once we know:

$$q^*(s, a)$$

We can choose:

$$\pi(s) = \arg\max_a q^*(s, a)$$

That gives the optimal policy.

How Monte Carlo estimates action values

Monte Carlo uses **episode returns**.

Whenever a **state-action pair** appears in an episode:

```
(s,a)
```

we record the **return that follows**.

Then we compute the **average return**.

Two Monte Carlo methods

1) First-Visit MC

2) Every-Visit MC

First-Visit vs Every-Visit for Action Values

- **Every-Visit MC**
- Average returns following **all visits** to (s, a)

- **First-Visit MC**
- Average returns following the **first occurrence** of (s, a) in each episode

Both converge to:

$$q_\pi(s, a)$$

as number of visits → ∞

- Monte Carlo:
  - Averages returns for state–action pairs
  - First-visit or every-visit
- Key challenge:
  - **Ensure exploration**

Exploration problem

Suppose policy π is deterministic.

Example:

```
State S
Policy always chooses action A1
```

Then during episodes we only see:

```
(S,A1)
```

We **never see**:

```
(S,A2)
(S,A3)
```

So we cannot estimate their values.

This is a **big problem** because we must compare actions to find the best one.

Practical solution:

Instead of deterministic policies, we use **stochastic policies**.

Example:

```
State S
Action probabilities

A1 → 0.7
A2 → 0.2
A3 → 0.1
```

Now **all actions have non-zero probability**, so they will eventually be explored.

This idea leads to:

- **ε-greedy policies**
- **soft policies**

Summary: Monte Carlo action –value

Monte Carlo action-value estimation

- Estimates $q_\pi(s, a)$, the expected return for state–action pairs.
- Uses the average of returns following visits to each pair.
- Two variants exist: **First-Visit MC** and **Every-Visit MC**.
- A key challenge is that some state–action pairs may never be visited.
- To ensure exploration, the **exploring starts assumption** or **stochastic policies** are used.

Monte Carlo Control -**how Monte Carlo methods are used to find the optimal policy** and how the practical **Monte Carlo ES (Exploring Starts) algorithm works**

- **Monte Carlo Control** is a reinforcement learning method used to **find the optimal policy** by learning from **complete episodes of experience**.
- It uses **Monte Carlo sampling** to estimate action values and **improves the policy repeatedly** until it becomes optimal.
- Monte Carlo control follows the concept of **Generalized Policy Iteration (GPI)**:
- Two processes run repeatedly:
  - **Policy Evaluation**
    Estimate how good the current policy is.
  - **Policy Improvement**
    Update the policy to choose better actions.

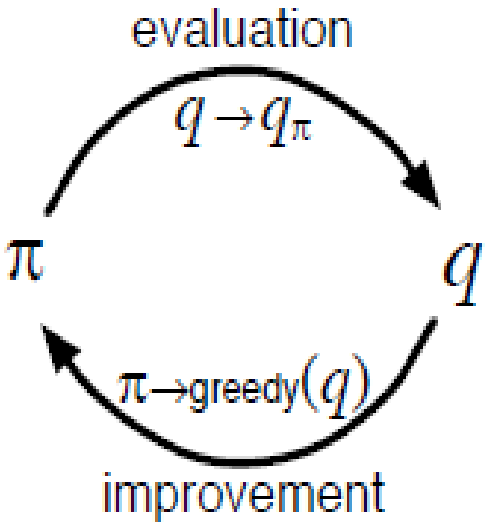
  This continues until the policy becomes **optimal**.

Algorithm Idea

```
Initialize policy π randomly
Initialize Q(s,a) arbitrarily

Repeat:
   Generate an episode using π
   For each state-action pair (s,a) in the episode:
       Update Q(s,a) using average return
   Improve policy:
       π(s) = argmaxa Q(s,a)
Until policy becomes stable
```

- **Monte Carlo Control** is a reinforcement learning method that learns the optimal policy by repeatedly estimating action-value functions from sampled episode returns and improving the policy using a greedy strategy. (important definition)



These two kinds of changes work against each other to some extent, as each creates a moving target for the other, but together they cause both policy and

value function to approach optimality.

To begin, let us consider a Monte Carlo version of classical policy iteration. In this method, we perform alternating complete steps of policy evaluation and policy improvement, beginning with an arbitrary policy π_0 and ending with the optimal policy and optimal action-value function:

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

where $\xrightarrow{\text{E}}$ denotes a complete policy evaluation and $\xrightarrow{\text{I}}$ denotes a complete policy improvement. Policy evaluation is done exactly as described in the preceding section. Many episodes are experienced, with the approximate action-value function approaching the true function asymptotically. For the moment, let us assume that we do indeed observe an infinite number of episodes and that, in addition, the episodes are generated with exploring starts. Under these assumptions, the Monte Carlo methods will compute each q_{π_k} exactly, for arbitrary π_k.

Policy improvement is done by making the policy greedy with respect to the current value function. In this case we have an *action*-value function, and therefore no model is needed to construct the greedy policy. For any action-value function q, the corresponding greedy policy is the one that, for each $s \in \mathcal{S}$, deterministically chooses an action with maximal action-value:

$$\pi(s) = \arg\max_a q(s, a). \tag{5.1}$$

To begin, let us consider a Monte Carlo version of classical policy iteration. In this method, we perform alternating complete steps of policy evaluation and policy improvement, beginning with an arbitrary policy π_0 and ending with the optimal policy and optimal action-value function:

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

where $\xrightarrow{\text{E}}$ denotes a complete policy evaluation and $\xrightarrow{\text{I}}$ denotes a complete policy improvement. Policy evaluation is done exactly as described in the preceding section. Many episodes are experienced, with the approximate action-value function approaching the true function asymptotically. For the moment, let us assume that we do indeed observe an infinite number of episodes and that, in addition, the episodes are generated with exploring starts. Under these assumptions, the Monte Carlo methods will compute each q_{π_k} exactly, for arbitrary π_k.

Policy improvement is done by making the policy greedy with respect to the current value function. In this case we have an *action*-value function, and therefore no model is needed to construct the greedy policy. For any action-value function q, the corresponding greedy policy is the one that, for each $s \in \mathcal{S}$, deterministically chooses an action with maximal action-value:

$$\pi(s) = \arg\max_a q(s, a). \tag{5.1}$$

Policy improvement then can be done by constructing each π_{k+1} as the greedy policy with respect to q_{π_k}. The policy improvement theorem (Section 4.2) then applies to π_k and π_{k+1} because, for all $s \in \mathcal{S}$,

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &= v_{\pi_k}(s). \end{aligned}$$

Policy improvement then can be done by constructing each π_{k+1} as the greedy policy with respect to q_{π_k}. The policy improvement theorem (Section 4.2) then applies to π_k and π_{k+1} because, for all $s \in \mathcal{S}$,

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s,a)) \\ &= \max_a q_{\pi_k}(s,a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &= v_{\pi_k}(s). \end{aligned}$$

As we discussed in the previous chapter, the theorem assures us that each π_{k+1} is uniformly better than π_k, or just as good as π_k, in which case they are both optimal policies. This in turn assures us that the overall process converges to the optimal policy and optimal value function. In this way Monte Carlo methods can be used to find optimal policies given only sample episodes and no other knowledge of the environment's dynamics.

Monte Carlo learns from **episodes**.

Example episode:

```
Start → Right → Up → Up → Goal
Reward = +10
```

From this episode we calculate the **return (G)**.

Then we update the value of the actions taken.

So the agent learns:

```
Q(state, action)
```

which means

👉 **how good it is to take action a in state s**

Suppose the agent always chooses the **same action**.

Example:

State S

Policy:

```
Always choose Right
```

Then the agent **never tries Left or Up**.

So it never learns their values.

Example:

```
Q(S,Right) → known
Q(S,Left) → unknown
Q(S,Up) → unknown
```

Then the agent **cannot know if Left or Up is better**.

This problem is called:

👉 **lack of exploration**

Monte Carlo ES(Exploring Starts)

- Monte Carlo ES solves **exploration problem.**

The idea is:

```
Each episode begins with a randomly selected state-action pair.
```

Example starting episodes:

```
Episode 1: start at (S2, A3)
Episode 2: start at (S1, A1)
Episode 3: start at (S4, A2)
```

This guarantees:

```
Every state-action pair will eventually be visited.
```

Thus we can estimate **Q(s,a)** for all pairs.

How ES works:

The algorithm alternates between **evaluation and improvement**.

Step 1: Initialize

Start with an arbitrary policy.

Step 2: Generate episode

Start from a random state–action pair (exploring start).

Step 3: Policy evaluation

Estimate action values:

$$Q(s, a) = \text{average of returns}$$

Step 4: Policy improvement

Update the policy:

$$\pi(s) = \arg\max_a Q(s, a)$$

Choose the action with the highest value.

Step 5: Repeat

Repeat until the policy stops changing.

Important:

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
    $Q(s,a) \leftarrow$ arbitrary
    $\pi(s) \leftarrow$ arbitrary
    $Returns(s,a) \leftarrow$ empty list

Repeat forever:
    Choose $S_0 \in \mathcal{S}$ and $A_0 \in \mathcal{A}(S_0)$ s.t. all pairs have probability > 0
    Generate an episode starting from S_0, A_0, following π
    For each pair s, a appearing in the episode:
        $G \leftarrow$ return following the first occurrence of s, a
        Append G to $Returns(s,a)$
        $Q(s,a) \leftarrow \text{average}(Returns(s,a))$
    For each s in the episode:
        $\pi(s) \leftarrow \arg\max_a Q(s,a)$

Figure 5.4: Monte Carlo ES: A Monte Carlo control algorithm assuming exploring starts and that episodes always terminate for all policies.

Monte Carlo Control without Exploring Starts

Exploring Starts

Meaning:

```
Start episodes at random state-action pairs
```

Example:

```
Episode 1 start → (S2, Up)
Episode 2 start → (S3, Left)
Episode 3 start → (S1, Down)
```

But this is **unrealistic** in real environments.

We cannot control the starting action.

New solution (ε-soft policy)

- Idea:

Step 1: Generate an episode

Follow the current policy.

Example:

```
S1 → A2
S2 → A1
S3 → A3
Goal
```

Step 2: Update action values

For each **state-action pair**, calculate the return.

Then update:

```
Q(s,a) = average return
```

Meaning:

```
How good was that action?
```

Step 3: Improve the policy

For each state choose the action with **highest Q value**.

Example:

```
Q(S,Left) = 5
Q(S,Right) = 8
Q(S,Up) = 6
```

Best action:

```
Right
```

So policy becomes **ε-soft greedy**:

```
Right → high probability
others → small probability
```

The algorithm repeats:

```
episode
→ update Q
→ improve policy
→ episode
→ update Q
→ improve policy
```

Gradually:

```
Q values become accurate
policy becomes better
```

Finally the agent learns **the best policy**.

- The **On-Policy First-Visit Monte Carlo Control algorithm with ε-soft policy** learns the optimal policy by repeatedly generating episodes, estimating action values using the average returns of first visits, and improving the policy to be ε-greedy with respect to the estimated action values. The process continues until the policy converges to the best policy.

Important

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s,a) \leftarrow$ arbitrary

$Returns(s,a) \leftarrow$ empty list

$\pi(a|s) \leftarrow$ an arbitrary ε-soft policy

Repeat forever:

(a) Generate an episode using π

(b) For each pair s, a appearing in the episode:

$G \leftarrow$ return following the first occurrence of s, a

Append G to $Returns(s,a)$

$Q(s,a) \leftarrow \text{average}(Returns(s,a))$

(c) For each s in the episode:

$a^* \leftarrow \arg\max_a Q(s,a)$

For all $a \in \mathcal{A}(s)$:

$$\pi(a|s) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & \text{if } a = a^* \\ \varepsilon/|\mathcal{A}(s)| & \text{if } a \neq a^* \end{cases}$$

Figure 5.6: An on-policy first-visit MC control algorithm for ε-soft policies.